

There are two more ways to create a Cron table. Usually, these methods are helpful when we want to invoke *crontab* via the shell script or when someone wants to specify the Cron job non-interactively. We can write the Cron job in a plain text file, and provide this file as a command line argument to *crontab* command as given below:

```
$ crontab -l
no crontab for tom
```

The following Cron job is stored in the plain text file.

```
$ cat tom.cron
# Check disk usage every week day at 11:30 PM.
30 23 * * 1-5 /home/tom/check_disk_usage.sh
```

Now specify the Cron job as a command line argument:

```
$ crontab tom.cron
```

Let us verify the Cron table's contents:

```
$ crontab -l
# Check disk usage every week day at 11:30 PM.
30 23 * * 1-5 /home/tom/check_disk_usage.sh
```

Additionally, we can also specify a Cron job inline without creating a Cron table as shown below:

```
$ crontab -l
no crontab for tom
```

Let us create a Cron job inline:

```
$ crontab << END_CRONTAB
> 30 23 * * 1-5 /home/tom/check_disk_usage.sh
> END_CRONTAB
```

Let us verify the Cron table's contents:

```
$ crontab -l
30 23 * * 1-5 /home/tom/check_disk_usage.sh
```

Experienced GNU/Linux users know the power of task automation. Isn't it a great idea to execute automated tasks without manual intervention? Given below are some useful Cron jobs that we can use in our day-to-day lives.

1) Installing kernel modules at system startup:

```
@reboot /home/tom/install_kernel_modules.sh
```

2) In the 'minute' field, if we specify an asterisk(*), then Cron will run the corresponding job every minute. But what if

someone wants to execute the Cron job only at a particular interval of time? For instance, to execute a Cron job every five minutes, add */5 in the 'minute' field:

```
*/* * * * * /home/tom/Check_download_is_completed.sh
```

3) The Cron job shown below will be executed at the 15th minute of every hour, on all days:

```
15 * * * * /home/tom/check_memory_usage.sh
```

4) There may be a requirement to execute a particular Cron job multiple times in a day. Let's say we need the build of software twice a day (only on working days) -- the first at 6 a.m. and the second at 7 p.m. For this, the Cron job can be written as:

```
00 6,19 * * 1-5 /home/tom/build_script.sh
```

5) Suppose you want send out monthly e-newsletters on the first day of the month; then the Cron job will look like:

```
0 0 1 * * /home/tom/monthly_news_letter.sh
```

Or:

```
@monthly /home/tom/monthly_news_letter.sh
```

6) We can also execute a Cron job only in a specific time range. For example, the following Cron job will schedule maintenance tasks only during weekends:

```
00 00 * * 6-7 /home/tom/weekly_maintenance.sh
```

So isn't Cron a simple yet great utility? Let us peep into the */etc* directory to dig out more about it. We know that for the individual user, there is a separate Cron table. Also, there is a systemwide Cron table defined in the */etc/crontab* file. Most of the time, this Cron table is used by the root user only. Unlike a user's *crontab*, this file has the 'user name' field specified for each command after the 'time' and 'date' fields and before the 'command' field. Typically, the systemwide Cron table has the following contents:

```
[root]# cat /etc/crontab
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-
```